

Adaptive Medical RAG Assistant

System Architecture, End-to-End Workflow, Data Pipeline & Key Functionalities Guide

1. Executive Overview & System Goals

The **Adaptive Medical RAG Assistant** is an evidence-based clinical decision support system designed to assist healthcare professionals with drug information, dosing adjustments, safety contraindications, and therapeutic alternatives. Traditional LLMs are prone to medical hallucinations; MedRAG mitigates this risk by grounding responses exclusively in authoritative clinical guidelines (e.g., GINA, GOLD, ADA, JNC) and official FDA prescribing labels (DailyMed).

Core Capabilities:

- **Evidence Grounding:** Generates structured clinical responses linked to exact document text passages and page numbers.
- **Patient-Specific Contextual Analysis:** Factors in patient age, renal function (eGFR), pregnancy status, comorbidities, and current medications.
- **Hybrid Retrieval Engine:** Combines dense vector semantics with sparse full-text lexical search using Reciprocal Rank Fusion (RRF).
- **Dual-Database Support:** Runs seamlessly on high-performance PostgreSQL (pgvector) or zero-config local SQLite.
- **Feedback & Audit Trail:** Logs all queries, patient parameters, retrieved evidence, and clinician ratings for audit compliance.

2. System Architecture & Component Layers

The application is structured into six modular layers, enforcing clean separation between ingestion, storage, retrieval, synthesis, and UI representation:

Layer	Component Modules	Primary Functionality
Presentation Layer	app/ui/gradio_app.py	Dual-panel clinician portal (Gradio Blocks) with forms for patient context, tabbed clinical summaries, citations, and feedback forms.
API Service Layer	app/main.py app/api/routes.py	FastAPI web service exposing endpoints (/health, /query, /evidence, /feedback) with Pydantic contract validation.
LLM Synthesis Layer	app/models/requests.py app/models/responses.py	Constructs RAG prompts with JSON Mode enforcement (GPT-4o-mini / Gemini), resolving numeric chunk indexes to verified database UUIDs.
Hybrid Retrieval Engine	app/retrieval/dense_retriever.py app/retrieval/sparse_retriever.py app/retrieval/hybrid_fusion.py	Performs parallel dense vector search (cosine distance) and sparse lexical search (FTS / ts_rank), merging ranks via Reciprocal Rank Fusion (RRF).

Ingestion Pipeline	app/ingestion/parsers/pdf_parser.py app/ingestion/section_chunker.py scripts/ingest_sources.py	Parses raw PDF files, scrubs NUL bytes, segments FDA labels into regulatory sections (warnings, contraindications, dosage), and chunks guidelines.
Storage & Database Layer	app/models/database.py app/db/session.py app/config.py	SQLAlchemy ORM models backing PostgreSQL (pgvector) and SQLite. Stores sources, documents, chunks, drugs, diseases, query logs, and audit trails.

3. End-to-End Information Processing Workflow

The system operates in two main operational phases:

Phase A: Ingestion & Indexing Pipeline (Offline Batch Processing)

- PDF Text Extraction:** `PDFParser` reads raw PDF documents page-by-page and strips illegal NUL (`0x00`) characters to prevent database errors.
- Regulatory Section Chunking:** `SectionChunker` uses regular expressions to detect standard FDA sections (*INDICATIONS*, *CONTRAINDICATIONS*, *WARNINGS*, *ADVERSE REACTIONS*, *DRUG INTERACTIONS*, *DOSAGE*) or page blocks for guidelines.
- Database Storage:** Saves source metadata, documents, and individual text chunks to the database.
- Vector Embedding Generation:** `create_indexes.py` computes 384-dimensional dense embeddings (`all-MiniLM-L6-v2`) or 1536-dimensional embeddings (`text-embedding-3-small`) and saves coordinates into `document_chunks.embedding`.

Phase B: Clinical Query & RAG Synthesis Workflow (Real-Time)

- User Query & Context Submission:** Clinician enters a query (e.g. *"Can a patient with severe renal impairment take Metformin?"*) along with patient age, eGFR, comorbidities, and medications via Gradio UI or REST API.
- Dense Semantic Search:** Vectorizes the question and queries `DocumentChunk.embedding` using PostgreSQL `<=>` cosine distance.
- Sparse Lexical Search:** Converts question into text search vectors (`to_tsquery`) and ranks matches using PostgreSQL `ts_rank`.
- Hybrid Rank Fusion (RRF):** Merges dense and sparse result sets using Reciprocal Rank Fusion: $RRF_score = \sum 1 / (60 + rank)$.
- Contextual Prompt Construction:** Assembles top candidate evidence chunks with chunk index numbers into a structured prompt.
- JSON Mode LLM Generation:** Queries GPT-4o-mini / Gemini using `response_format={'type': 'json_object'}`, enforcing a strict clinical schema.
- Citation Resolution & Audit Logging:** Maps numeric `chunk_index` values in citations back to verified database chunk UUIDs and source passages. Writes query log and audit trail to database.

4. Hybrid Evidence Retrieval & RRF Reranking Algorithm

RAG applications relying solely on vector embeddings often miss exact drug name matches or specific numerical dosages. Conversely, keyword search fails on complex semantic phrasing. MedRAG combines both approaches:

Reciprocal Rank Fusion (RRF) Formula:

For each document chunk d appearing in the retrieved sets R_{dense} and R_{sparse} :

$$RRF_Score(d) = (1 / (k + Rank_{dense}(d))) + (1 / (k + Rank_{sparse}(d)))$$

Where constant $k = 60$ (standard RRF smoothing factor).

Dialect Portability: When running on PostgreSQL, `DenseRetriever` uses native `pgvector` cosine distance and `SparseRetriever` uses PostgreSQL Full-Text Search. When running on SQLite, fallback mechanisms use tokenized case-insensitive substring matching and metadata filters.

5. REST API Endpoints & Response Schema Contract

The FastAPI backend (`app/api/routes.py`) provides the following endpoints:

HTTP Method & Endpoint	Input Payload / Params	Description & Output
GET /health	None	Returns API operational status, server timestamp, app name, and environment.
POST /api/v1/query	QueryRequest JSON (question, patient_context, jurisdiction)	Executes RAG search, generates structured QueryResponse JSON, and records QueryLog + AuditLog entries.
GET /api/v1/query/{id}/evidence	query_id (UUID string)	Retrieves all underlying document text chunks, page numbers, and section labels linked to a query.
POST /api/v1/feedback	FeedbackRequest JSON (query_id, rating, citation_quality, comment)	Logs clinician evaluation ratings (helpful/unhelpful) and comments into Feedback database table.

Structured Response Contract (QueryResponse JSON):

- summary:** Direct 1-2 sentence clinical answer.
- drug_information:** Generic name, brand names, class, active ingredients, mechanism.
- indications:** Approved indications vs. evidence-supported off-label uses.
- patient_considerations:** Age, pregnancy/lactation, renal, hepatic, allergies, comorbidities.
- contraindications & warnings:** Absolute contraindications and FDA boxed warnings.
- side_effects:** Common vs. serious adverse reactions.
- treatment_timeline & monitoring:** Expected onset, reassessment period, and required lab monitoring.
- alternatives:** Generic equivalents, same-class options, and guideline-supported options.
- evidence_indicators:** Evidence quality score, population match, source agreement, completeness.
- citations:** List of claims linked to exact chunk IDs, passages, and document URLs.

6. System Execution & Operational Commands

To start the system locally, execute the following commands in order:

```
# Step 1: Boot up PostgreSQL + pgvector Docker container
docker compose up -d

# Step 2: Run Alembic database migrations
```

```
.venv\Scripts\alembic upgrade head

# Step 3: Parse PDF guidelines and drug labels into database
.venv\Scripts\python scripts/ingest_sources.py

# Step 4: Calculate vector embeddings & create pgvector index
.venv\Scripts\python scripts/create_indexes.py

# Step 5: Launch FastAPI Backend Service
.venv\Scripts\python app/main.py

# Step 6: Launch Gradio Clinician Web Interface (http://localhost:7860)
.venv\Scripts\python app/ui/gradio_app.py
```

Database Verification: You can verify database tables, chunk counts, and raw vector coordinates at any time by running:

```
.venv\Scripts\python scripts/inspect_database.py
```